

AA-01

Repository Topology, CI/CD and the Shared Portal Kit

Five repositories, their boundaries, pipelines, and the versioned kit between them

30 August 2026 · Rev 1.0 — baseline

REPOSITORIES	PUBLISHED PACKAGES	CI GATES
Five	Two	Twelve
CONVENTIONS	PORTAL ENV VARS	RECURRING COST
Twelve	Six	Rs 0

This document answers one question: where does each line of code live, and how does it reach production. It is the second of thirteen and it assumes AA-00. The repository split itself is decided in AA-00 section 0.3.1; here it is argued, costed and made operational. Section 1.2 is the honest section: it states what the five-repository split costs us, names the failure mode it introduces, and admits that the mitigation is only a mitigation.

1.1 WHAT THIS DOCUMENT DECIDES

Five repositories, and the one question they answer

There is exactly one question this document exists to answer, and every section below is a restatement of it: where does each line of code live, and how does it reach production. Get that wrong and four engineers spend ten weeks stepping on each other, a secret ends up in a browser bundle, and nobody can say which commit is running on which hostname.

The answer is five repositories. One backend, one shared front-end package, and one repository per portal. Two npm packages are published out of that set: the API contract, generated from the backend, and the portal kit, hand-written by FE-1. Everything else is a consumer.

REPOSITORY	KIND	STACK	DEPLOY TARGET	OWNER	RESPONSIBILITY
atavishaala-api	Backend	Express 5 · TS · Prisma · Node 20	Google Cloud Run (asia-south1)	BE-1, BE-2	The only writer to Postgres. Modular monolith, 11 modules, 4 API namespaces. Publishes @atavishaala/contract on tag.
atavishaala-portal-kit	Shared package	React 18 · TS · Vite lib mode · Tailwind preset	GitHub Packages (private npm)	FE-1 owns, FE-2 reviews	Design system, session/auth hooks, permission gate, PII masking primitives, generated API client re-export. Ships @atavishaala/kit.
atavishaala-admin-portal	Front end	Vite · React 18 · TanStack Query · Router 6	Cloudflare Pages -> admin.atavishaala.in	FE-1	Internal staff console. 41 screens including the agronomist Advisor console. Consumes /api/v1/admin only.
atavishaala-supplier-portal	Front end	Vite · React 18 · TanStack Query · Router 6	Cloudflare Pages -> seller.atavishaala.in	FE-2	Product-company self-serve. 26 screens. Consumes /api/v1/partner only.
atavishaala-fleet-portal	Front end	Vite · React 18 · TanStack Query · Router 6	Cloudflare Pages -> fleet.atavishaala.in	FE-2	Delivery-company self-serve, mobile-first for riders. 24 screens. Consumes /api/v1/fleet only.

The load-bearing line in that table is the first one. atavishaala-api is the only process in the system holding a database credential, and therefore the only writer to Postgres. The three portals hold no DATABASE_URL, no Prisma client, no Cloudinary API secret, no Razorpay secret and no JWT signing key. They hold six environment variables, all of them public by construction, and section 1.8 lists them.

This is not a stylistic preference about where logic should live. A Cloudflare Pages bundle is a static asset served to anyone who asks for it. Any value compiled into that bundle is public the moment the first deploy completes, whether or not anyone has noticed. So the rule is phrased as an absence rather than a discipline: there is no code path in a portal repository that could use a database credential, because there is no database client in its dependency tree. Invariant I-01 states it and the secret-scan and no-prisma-in-portal gates enforce it.

WHAT A PORTAL IS ALLOWED TO DO

Render, validate for the user's benefit only, and make authenticated HTTPS calls to api.atavishaala.in through the kit client. Nothing else. A portal may reject a form before submitting it; the API rejects it again and the API is the one that counts.

WHAT A PORTAL IS NOT ALLOWED TO DO

Hold a secret, reach a third-party API directly, own a business rule that writes, or import anything that speaks to Postgres. A bare fetch or an axios import fails the aa/no-direct-http rule before review.

1.2 WHY FIVE REPOSITORIES AND NOT ONE

The argument, the rejected alternatives, and what the choice costs us

A monorepo is the better answer at most companies and it is the answer I would give with eight front-end engineers and a build cache. It is not the answer here, and the reasons are specific to this programme rather than general. They are worth stating properly, because in week six somebody will be annoyed enough by a version bump to propose collapsing the set, and that person deserves an argument rather than a policy.

Three alternatives were considered seriously. Each is written below with the reason it lost.

#	REJECTED ALTERNATIVE	WHY IT LOST
A	One monorepo with pnpm workspaces and Turborepo. All five packages in one tree, one lockfile, one CI workflow, task graph caching.	Two reasons, and the second is the serious one. First, the three portals have different release cadences and three different Cloudflare Pages projects: admin ships when ops needs a fix, the supplier portal ships on the supplier onboarding schedule, and the fleet portal ships when riders are not mid-shift. A monorepo makes independent deploys possible but not natural, and the path-filter configuration that keeps them independent is itself a thing that breaks silently. Second, in one tree it is trivially easy for a portal to import a server-only module. A single import of a shared config file that reads process.env pulls a secret name into the Vite graph, and Vite will inline any VITE_ prefixed value it finds. The failure is a compiled secret, discovered by whoever reads the bundle. Separate repositories make that import impossible rather than merely discouraged, which is the only kind of prevention worth relying on with four engineers and no dedicated reviewer.
B	One repository per portal and no shared kit. Three independent front ends, each with its own components, its own client and its own auth handling.	Three copies of the same four things: the 401 refresh interceptor with its single-flight lock, the pause-to-rupee formatter, the masked-PII cell, and the step-up TOTP modal. Those four are exactly the code where a divergence is a security or a money bug rather than a cosmetic one. The observed pattern is that copies drift within a month: one portal fixes a double-refresh race and the other two do not, or one formats 1050 paise as Rs 10.50 and another as Rs 10.5. Risk R-11 names this directly. Duplicating a button is fine; duplicating an auth interceptor is not.
C	Fold the admin console into the existing customer SPA repository, since the tooling and the design tokens are already there.	The customer SPA and the admin console have opposite threat models. The customer surface is anonymous-first, wide open, cached at the edge, and its worst case is a wrong price. The admin console is authenticated-only, TOTP-gated, PII-adjacent, audited on every mutation, and its worst case is a data breach. Sharing a repository means sharing a CSP, a dependency set, a review bar and a deploy trigger across those two postures. It also means an admin route becomes reachable by URL from the customer origin, which is the sort of thing that is fine until a route guard regresses.

1.2.1 The cost: the version-skew tax

The split is not free and the bill arrives as version skew. Four repositories consume two published packages at exact versions, which means at any moment the three portals may be running three different versions of the kit and of the contract. That is the tax. It is paid in small, irritating increments and occasionally in one embarrassing bug.

Here is the concrete failure mode, written out because an abstract warning gets ignored. The admin portal is on @atavishaala/kit@1.4.0. The fleet portal is on @atavishaala/kit@1.2.3, because FE-2 was mid-sprint on the offer inbox when 1.3.0 landed and did not want a dependency change in that pull request. In 1.3.0 the kit added three entries to the error-code map: OFFER_TAKEN, RIDER_AT_CAPACITY and DISCLOSURE_WINDOW_CLOSED. The API, which is on one version because there is only one of it, starts returning RIDER_AT_CAPACITY when a dispatcher attaches a rider already holding eight jobs. The admin portal renders "This rider is already at the eight-job limit." The fleet portal, whose kit has no entry for that code, falls through to its default branch and renders the raw string RIDER_AT_CAPACITY to a dispatcher in a control room. Nothing crashed, nothing was logged as an error, and the only signal is a support message asking what RIDER_AT_CAPACITY means.

That is the shape of every skew bug: the newer consumer is correct, the older consumer is quietly wrong, and the wrongness surfaces to a user rather than to a test. It is worse for the contract package than for the kit, because a stale contract does not degrade gracefully. If the API adds a required field to a Zod request schema and the fleet portal is still on the previous contract, every write from that portal returns 400 with a field error the portal cannot map, and the portal is functionally down for writes while looking healthy.

STATE THE TAX PLAINLY

A monorepo has one version of everything by construction, and that is a real advantage we are giving up. We are trading a guaranteed-consistent dependency graph for a guaranteed-separate deploy and build boundary. That trade is right for this team at this size. It would be the wrong trade at three times the team, and the row in AA-00 section 0.15 is where to reopen it.

1.2.2 The mitigation, and why it is only a mitigation

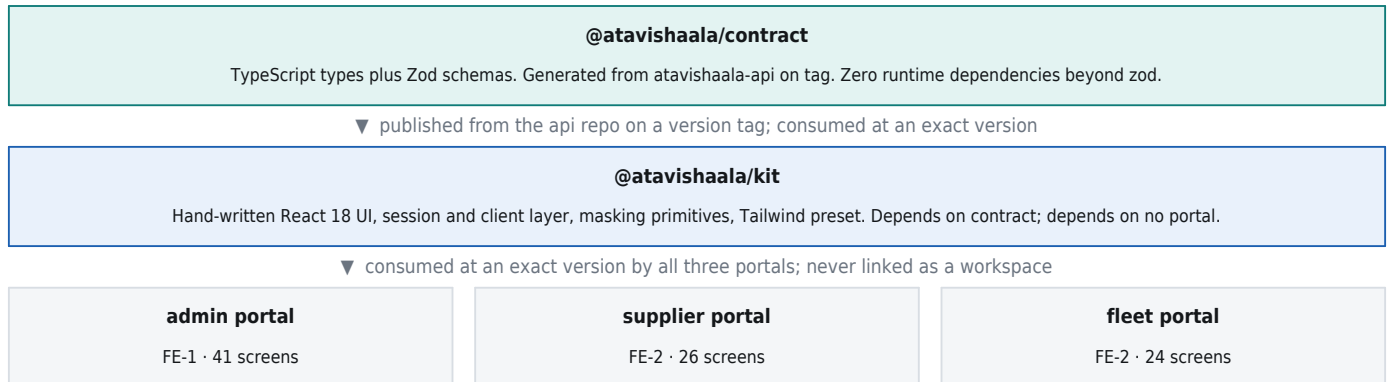
MITIGATION	MECHANISM	WHAT IT DOES NOT COVER
A weekly kit-bump chore	The kit release for the week ships at the end of the Wednesday 18:00 blocker sync. On Monday 09:00 each front-end engineer states their current kit version out loud as part of the kick-off, alongside their task IDs.	A chore is a habit, and habits fail in week seven when settlement is late. Nothing in CI notices that the chore was skipped, only that the gap has grown.
A CI gate on the minor gap	Each portal build reads the published latest of @atavishaala/kit from GitHub Packages and fails if the pinned minor is more than one behind. One behind is tolerated so an in-flight pull request is not broken by somebody else's release; two behind is a hard stop.	It catches distance, not correctness. A portal exactly one minor behind can still render a raw error code for the whole week, which is precisely the case in the example above.
Additive-only inside a major	A kit minor may add an export, add an optional prop, or add an error-code entry. It may not remove an export, rename a prop, change a default or change a rendered format. Anything in that second list needs a major bump and three coordinated portal pull requests in the same week.	It relies on the author correctly classifying their own change. A prop default quietly changed from false to true is additive on paper and breaking in practice.
Exact pinning, no ranges	Invariant I-13. No caret, no tilde, no workspace link in a portal package.json for either package. The range-pin CI gate parses each portal manifest and fails on a range specifier.	Pinning makes skew visible and reproducible rather than absent. It is the opposite of a fix: it guarantees the skew exists until somebody bumps it. That is deliberate, because the alternative is a caret range silently changing behaviour on a rebuild.

None of those four prevents the failure. Together they bound how long it can persist and make it cheap to diagnose: when a portal renders something wrong, the first question is what kit version it is pinned to, and the answer is one line of its package.json and one line in its footer via VITE_BUILD_SHA. That is the honest claim. The skew bug will happen at least once in ten weeks; the goal is that it takes ten minutes to identify rather than an afternoon.

1.3 THE DEPENDENCY GRAPH

Two published packages, one direction, no cycles

The graph is deliberately a tree with two levels above the leaves, and every edge points downward. There is no edge from a portal to another portal, no edge from the kit back to a portal, and no edge from the contract to the kit.



1.3.1 Why two packages and not one

The obvious simplification is a single package holding both the types and the components. It was rejected because the two halves have different authors, different change rates and, most importantly, different notions of correctness.

The contract is generated. Its source of truth is the Zod schemas in atavishaala-api, and the generation runs in the api repository on a version tag. Nobody hand-edits it, and the convention in AA-11 Appendix D is explicit that hand-editing the generated OpenAPI document or client is forbidden. That is what makes drift between the API and its declared types impossible rather than unlikely: there is no second place where the shape is written down, so there is nothing to disagree with. AA-02 covers how the schemas are organised inside the api repository and AA-04 is the frozen surface itself.

The kit is the opposite. It is hand-written, it is judged by how a screen looks and behaves, and it changes on a weekly release cadence driven by whatever FE-1 and FE-2 needed that week. Merging it with a generated artefact would mean either regenerating the whole package on every component tweak, or tagging the api repository every time a button gains a size prop. Both are worse than two packages.

WHY THE KIT DEPENDS ON THE CONTRACT

Because the kit is where typed data becomes pixels. The Money component takes a value the contract has already typed as integer paise. The useApi wrapper is generic over the contract's request and response types, so a portal calling an endpoint gets its response shape without declaring anything. The error-code map is keyed by the contract's error-code union, which is how a missing entry becomes a type error rather than a raw string on screen once both packages are current.

WHY THE REVERSE EDGE IS FORBIDDEN

A dependency from contract to kit would mean the backend repository, which generates the contract, transitively depends on React. That drags a UI dependency tree into a Cloud Run image, makes the api build order depend on a front-end release, and creates a cycle that no version pinning scheme can resolve. It is enforced by the contract package having exactly one dependency, zod, and a CI check on that fact.

THE RULE IN ONE LINE

Types flow out of the backend. Components flow out of the kit. Nothing flows back up. A portal that needs a shape the contract does not export is asking for an API change, not a local type declaration, and a local type declaration duplicating a contract shape is a review rejection.

1.4 HOSTNAMES AND EDGES

One Pages project per repository, three subdomains, one exact CORS list

HOSTNAME	SERVES	INFRASTRUCTURE	AUDIENCE
api.atavishaala.in	Backend API	Cloud Run + Cloudflare proxy	all four audiences
admin.atavishaala.in	Admin portal	Cloudflare Pages	StaffUser only
seller.atavishaala.in	Supplier portal	Cloudflare Pages	PartnerUser only
fleet.atavishaala.in	Fleet portal	Cloudflare Pages	FleetUser only
www.atavishaala.in	Marketing + legal	Cloudflare Pages	public
cdn (res.cloudinary.com)	Media delivery	Cloudinary free	public, signed uploads

Each portal repository maps to exactly one Cloudflare Pages project, and each Pages project maps to exactly one custom domain. That one-to-one-to-one chain is what makes a deploy traceable: a commit on main in atavishaala-fleet-portal produces one build, in one project, reaching one hostname. A pull request in the same repository produces a preview deployment on a pages.dev URL, which is where a reviewer looks before approving, and which is not in the CORS allow-list, so a preview build cannot talk to the production API.

The three surfaces are subdomains, not path prefixes on one origin. That was a real decision and it has two reasons, both of them security rather than aesthetics.

WHY NOT ONE ORIGIN WITH PATH PREFIXES	THE CONSEQUENCE OF THE SUBDOMAIN CHOICE
Cookie scope. The refresh cookies are host-only, Secure, HttpOnly and SameSite=Strict, named atv_admin, atv_partner and atv_fleet. On one origin, a cookie set at the origin is sent on every path by default, so a partner session cookie would be presented to /admin paths and a path-scoped cookie would be the only defence. Path scoping on cookies is not a security boundary, it is a hint. Host-only cookies on three hostnames are a boundary the browser enforces.	A stolen partner refresh cookie is useless against admin.atavishaala.in because the browser will not send it there and the API would reject the audience anyway. Two independent mechanisms, neither relying on the other. AA-08 covers the cookie and token posture in full.
WAF rule separation. The free Cloudflare plan allows five custom WAF rules. With three hostnames each rule can be scoped to a host, so a rate limit or a country block on the admin console does not apply to the fleet portal that riders hit from mobile networks. On one origin every rule is a path-matching rule, and path matching in a WAF expression is where mistakes live.	The admin host can carry a stricter rule set than the fleet host without a shared budget argument, and the five-rule ceiling is spent on host-scoped rules rather than on path discrimination. AA-09 records the rule budget as a free-tier line item with a named threshold.

On the other side of the edge, the API decides who may call it. ALLOWED_ORIGINS in the api repository environment is an exact list of the four production origins plus localhost for development, and it is read as a list, not a pattern. There is no wildcard, no suffix match on atavishaala.in and no reflect-the-Origin-header behaviour, because a suffix match on a domain that can host a Pages preview is the same thing as no check at all. An Origin that is not on the list is refused with 403 before any handler runs, which is task P1-BE1-04.

The practical cost is that adding a portal or a preview host means an environment change and a redeploy of the API rather than a front-end change. That is the intended friction. The CORS list is a security control and it lives with the process that enforces it, not in a header file next to the thing being controlled.

1.5 INSIDE THE PORTAL KIT

What belongs in the kit and, more usefully, what does not

The kit is owned by FE-1 and reviewed by FE-2, and FE-2 has direct commit rights on it. That second clause matters more than it looks: the failure mode for a shared package on a small team is not that it is badly designed, it is that the person who needs a component cannot get it merged this week and copies it into their portal instead. Risk R-05 tracks exactly that, and its detection signal is a new file under a portal components directory whose export name duplicates a kit export.

The boundary is drawn by one test. If two portals would need it and it would be a bug for them to differ, it belongs in the kit. If it describes what this particular portal is, it does not.

IN THE KIT — SHARED BECAUSE DIVERGENCE WOULD BE A BUG	NOT IN THE KIT — PORTAL-SPECIFIC BY DEFINITION
Auth session provider and the 401 refresh interceptor, with a single-flight lock so six concurrent 401s produce one refresh call.	Any route table. Routes are what a portal is; a shared route table would be a shared identity.
useApi, a typed fetch wrapper generic over the contract request and response types. The only way a portal reaches the network.	Any permission string. The kit ships the Can gate and useCan; the strings live in the portal that uses them and ultimately in the backend union.
Money, a component that accepts integer paise and nothing else. There is no prop that takes a rupee value or a float.	Any business copy. A kit that holds sentences becomes the place product wording is negotiated, and then every copy change is a package release.
Masked, a component that cannot render more than four characters of a value.	Any tenant-specific or organisation-specific branding. There is one brand here and it is in the Tailwind preset, not in per-tenant theming that nobody asked for.
DataTable with server-side pagination, column visibility and a sticky header.	Any direct fetch to a hard-coded host. The base URL comes from VITE_API_BASE at the portal edge and is passed into createClient, so the kit never knows a hostname.
FilterBar producing a typed query object, Drawer, and ConfirmDialog with a typed reason field.	Any screen. A screen composes kit parts; the moment a screen is in the kit, two portals start adding conditionals to it.
StepUpDialog, which intercepts 403 STEP_UP_REQUIRED, collects a fresh TOTP and replays the original request without losing form state.	
EmptyState, because an empty list that names the next action is the difference between a usable portal and a confusing one.	
The error-code to human-string map, keyed by the contract error-code union.	
IST date and time formatting. Store UTC, compute business hours in Asia/Kolkata, never trust a client clock for an SLA.	
The Idem hook, which mints an idempotency key per user intent and retains it across retries so a retry is a replay and not a second write.	

1.5.1 Masked is a type-level guarantee, not a convention

The programme rule is that no portal ever renders more than the last four characters of a phone number, an email local part, an account number or a UPI handle, and that there is no reveal affordance anywhere in any of the three portals. A convention cannot hold that line. Somebody will write a tooltip, or a CSV export, or a debug panel, and it will pass review because the reviewer was looking at the query and not at the cell.

So the kit does not ship a masking function that a developer may or may not call. It ships a component whose props cannot express the unmasked case. The value arrives already reduced to four characters by the API — the backend serialiser never emits the full string — and the component types accept only that shape.

```
// @tavishaala/contract — the wire shape. There is no `full` field to leak.
export type MaskedValue = { readonly last4: string; readonly kind: 'phone'|'email'|'acct'|'upi' };

// @tavishaala/kit — the only component permitted to render a MaskedValue.
export function Masked(p: { value: MaskedValue; label?: string }): JSX.Element;
//   p.value.last4 is sliced to 4 chars at render time regardless of what arrives.
//   There is no `reveal`, no `showFull`, no `onReveal`. Adding one is an API change,
//   which means a kit major bump, which means a review nobody can merge past quietly.

// ESLint rule aa/no-raw-pii-render (blocking, gate 04):
//   flags any JSX expression whose identifier matches /phone|email|account|upi|aadhaar/i
//   and is not wrapped in <Masked>. Escape hatch: none. Suppression comment: fails CI.
```

This is worth being precise about, because it is the difference between a control and a hope. Three things have to fail together for a full phone number to reach a screen: the serialiser would have to emit it, the contract type would have to be regenerated to carry it, and the lint rule would have to be suppressed. Any one of those alone is caught. The single exception in the whole programme is GET /fleet/jobs/:id/manifest, which releases the

delivery address only, scoped to one job and to a window of `MANIFEST_WINDOW_MINUTES = 240`, and writes a `pii_disclosures` row in the same transaction as the read. The customer phone is never released by that endpoint or any other. AA-08 carries the full disclosure contract.

THE KIT DOES NOT MAKE PII SAFE ON ITS OWN

Masked stops an accidental render. It does nothing about a screenshot, a support agent reading a value aloud, or a database export. Those are handled elsewhere or not at all, and AA-08 says which is which. Do not read this section as a claim that the PII problem is closed.

1.6 BRANCH, COMMIT AND REVIEW CONVENTIONS

Twelve rules, all of them enforceable, most of them enforced by a human

These twelve are the whole of the convention set. There is no style document beyond this table and the Prettier config, deliberately: a four-developer team on a ten-week programme cannot afford a convention it has to look up. Each row below is either checked by CI (section 1.7) or checked by the reviewer named in the Reviews row, and nothing else is a convention at all.

AREA	RULE
Branches	<code>feat/<area>-<short></code> , <code>fix/<area>-<short></code> , <code>chore/<area>-<short></code> . No personal branches.
Commits	Conventional commits. The scope is the module or the portal, e.g. <code>feat(fleet): offer wave sweep</code> .
PRs	One task ID per PR, named in the title. A PR that touches two repos is two PRs with a cross-link.
Reviews	BE-1 reviews BE-2 and the reverse. FE-1 reviews FE-2 and the reverse. Kit changes need both FE reviews.
Migrations	One migration per PR. Never edit an applied migration. Every migration has a rollback note in the PR body.
Contracts	Zod schema first, then generate. Never hand-edit the generated OpenAPI or client.
Kit changes	Additive within a minor. A breaking kit change needs a major bump and a coordinated portal PR.
Feature flags	Any feature that spans a week boundary ships behind a flag defaulting to off.
Error codes	<code>SCREAMING_SNAKE</code> , defined once in <code>packages/shared/errors.ts</code> . A new code needs a line in AA-04.
Time	Store UTC. Compute business hours in Asia/Kolkata. Never trust a client clock for an SLA.
Money	Integer paise everywhere. Format for display only, at the very edge, in the kit.
Logging	Pino structured JSON. Never log a masked field, even masked. Log identifiers, not values.

Two rows carry more weight than the rest. "One task ID per PR" is what makes AA-11 usable as a tracker at all — the week plan is a list of task IDs, and if a PR spans two of them the plan stops reflecting reality within three days. The cross-repo corollary is the awkward part: a change that adds an error code to the contract and consumes it in the fleet portal is two PRs, merged in that order, and the second one cannot go green until the first has published. That is the version-skew tax of section 1.2.1 showing up as day-to-day friction rather than as an incident, which is the trade we chose.

The other is "Kit changes need both FE reviews". FE-1 owns the kit and FE-2 can commit to it, so on paper either could ship alone. The rule exists because the kit is the only artefact in the programme where a single merge changes three products, and because the pair of them are the only two people who will notice that a prop rename is breaking rather than additive.

WHY CONVENTIONAL COMMITS, GIVEN NOTHING CONSUMES THEM

Nothing generates a changelog from the log — the `CHANGELOG.md` entry is hand-written and CI checks it exists (section 1.9). The commit convention is kept anyway because the scope token makes `git log --oneline` readable per module during a Wednesday blocker sync, and because a `grep` for `feat(fleet)` is the fastest way to reconstruct what week 6 actually shipped.

WHY NO PERSONAL BRANCHES

A branch named `ayushman/wip` cannot be picked up by anybody else when its author is on a dental posting. Branch names carry the area and the intent so that a stalled branch is legible to the other three developers. This is a small team convention with a specific cause, not hygiene for its own sake.

1.7 THE CI PIPELINE

Twelve gates, what each one is defending, and what it costs in minutes

Every repository runs the same workflow file shape: install with a cached lockfile, then the gates that apply to that repository kind. Ten of the twelve are blocking — the merge button is disabled until they pass — and the two that are advisory are marked as such in the walkthrough below. The list is deliberately short. Each gate exists because a specific invariant in this programme is otherwise unenforceable by review alone.

GATE	APPLIES TO	WHAT IT ASSERTS
Typecheck	all five repos	tsc --noEmit. Zero errors. No // @ts-expect-error without a linked issue.
Lint	all five repos	eslint with the eight custom rules above as errors, not warnings.
PII grep	all five repos	A pre-commit and CI grep for ten consecutive digits in a template literal or JSX text.
Secret scan	all five repos	gitleaks over the full history, not just the diff.
Range pin check	three portals	package.json must pin @tavishaala/kit and /contract to an exact version.
Contract sweep	atavishaala-api	Every non-public route asserts 401 for the three wrong token audiences.
Permission sweep	atavishaala-api	Every admin route declares a permission that exists in the union.
Idempotency replay	atavishaala-api	Every money and offer fixture is replayed; the second response must be byte-identical.
FSM table test	atavishaala-api	All 28 shipment-state combinations drive reduceOrderStatus to the expected value.
Migration rehearsal	atavishaala-api	M1..M8 applied to a restored Neon branch, then a derived-status diff of zero rows.
CSP check	three portals	Playwright asserts the deployed CSP header has no unsafe-inline and no wildcard connect-src.
Bundle budget	three portals	Initial JS under 220 KB gzipped per portal. Fails the build above it.

Three of these are worth naming as unusual. The secret scan runs over full history rather than the diff, because a secret that was committed in week 2 and removed in week 3 is still a secret in the clone every developer has. The range pin check exists because l-13 requires exact version pins for the kit and the contract — a caret range would let a portal pick up a kit minor on the next clean install and make the skew in section 1.2.1 non-deterministic, which is strictly worse than skew you can read off package.json. And the idempotency replay asserts byte-identical second responses rather than merely a 200, because a replay that returns a different payment reference is a duplicate write wearing a success code.

1.7.1 One pull request, end to end

Take P1-FE2-03 in the fleet portal: a PR that adds the offer-acceptance drawer and consumes a new contract type. FE-2 pushes to fix/fleet-offer-drawer. What happens, in order, with observed wall-clock times on the free runner:

ELAPSED	STAGE	DETAIL	MODE
0:00	Push, workflow queued	Free-tier queueing on ubuntu-latest is usually under 20 seconds and has been as long as 4 minutes on a Monday morning. This is the one number nobody can control.	n/a
0:25	Install	actions/setup-node with the npm cache keyed on package-lock.json. Warm cache: 18 s. Cold cache, which happens whenever the lockfile moves: 55 s.	Blocking
0:45	Typecheck	tsc --noEmit. This is where a stale contract pin surfaces first, as a missing member on the error-code union rather than as a runtime 400.	Blocking
1:05	Lint plus PII grep	eslint with the eight custom rules as errors. The PII grep runs in the same job because it is a riggrep pass and costs under a second.	Blocking
1:20	Range pin check	A node script reading package.json. Rejects ^, ~, workspace: and file: for the two @tavishaala packages.	Blocking
1:25	Secret scan	gitleaks over full history. Roughly 30 s at this repository size and grows slowly with commit count.	Blocking
1:55	Unit and component tests	Vitest with jsdom. The fleet portal suite is 40 s today; the budget assumption is that no portal suite exceeds 90 s.	Blocking

ELAPSED	STAGE	DETAIL	MODE
2:35	Build plus bundle budget	vite build, then a size check against 220 KB gzipped initial JS. A PR that adds a chart library fails here rather than in review.	Blocking
3:10	Cloudflare Pages preview	Pages builds the branch independently of Actions and posts a preview URL. It does not consume Actions minutes; it consumes one of the 500 monthly Pages builds.	Blocking
4:30	CSP check	Playwright hits the preview URL and asserts the response CSP has no unsafe-inline and no wildcard connect-src. Advisory on a branch, blocking on main.	Advisory
5:00	Bundle-delta comment	A bot comment reporting the gzipped delta against main. Informational; it never fails a build, because a 3 KB regression is not worth blocking a merge over.	Advisory

So a clean portal PR is green in about five minutes and costs roughly three Actions minutes against the budget. An api PR is longer: the same front half, then the contract sweep, the permission sweep, the idempotency replay and the 28-case FSM table test, which together run about three minutes, and on any PR containing a migration a rehearsal against a restored Neon branch that adds another two. AA-02 covers what those api-side suites actually assert.

1.7.2 The minute budget, and what happens when it stops fitting

GitHub Actions is unmetered on public repositories and capped at 2 000 minutes per month on private ones. All five of these repositories are private, because the api repo holds the permission union and the migration set and the portals hold the admin surface, so the unlimited-on-public escape does not apply. The programme therefore has 2 000 minutes a month for five repositories over ten weeks, with an alert at 1 400.

That fits, but only because the per-run costs are small. The arithmetic is worth writing down because it is the assumption that breaks first.

CONSUMER	RUNS PER MONTH	MINUTES EACH	SUBTOTAL
Three portal repos, PR and merge runs	about 130	about 3.0	about 390 min
atavishaala-api, PR and merge runs	about 70	about 6.5	about 455 min
atavishaala-api, migration rehearsal runs	about 20	about 2.0	about 40 min
atavishaala-portal-kit, publish runs	about 20	about 2.5	about 50 min
Scheduled workflows that are not Cloud Run crons	about 60	about 1.0	about 60 min
Headroom before the 1 400 alert			about 405 min

Two things would break this. A Playwright end-to-end suite that boots a real browser against a preview URL on every PR costs four to six minutes a run, which on 200 runs a month is 1 000 minutes on its own and the budget is gone by week 6. A matrix build across Node versions would do the same for no benefit, since there is exactly one runtime target. Neither is in the plan.

If the budget is exhausted anyway, the degradation is not graceful and should be understood in advance. Actions stops running for the rest of the billing month on all five repositories at once; it does not slow down or queue. Merges are then unguarded, because branch protection requires checks that can no longer run, and the team either merges without gates or waits for the reset. The stated response, in order: move the scheduled workflows to the Cloud Run cron runner where they belong anyway, drop the CSP check and the bundle-delta comment to a nightly run on main only, and if it is still tight, run the api migration rehearsal on demand via workflow_dispatch rather than on every migration PR. The last of those trades a real safety net for minutes and is the point at which paying for Actions minutes should be reconsidered against the Rs 0 constraint. AA-09 holds the free-tier ledger and the alert thresholds for every provider, including the 500-builds-per-month Cloudflare Pages line that the preview deployments draw against.

THE GATES ARE ONLY AS GOOD AS BRANCH PROTECTION

A required check that is not marked required in the branch protection rule is decoration. Marking the ten blocking gates required is a manual, per-repository step on the free tier — there is no organisation-level ruleset to inherit from. It is step 4 of the bootstrap in section 1.10, and it is the step most likely to be skipped on a new repo.

1.8 ENVIRONMENT VARIABLES FOR A PORTAL

Six values, all six public, and the gate that keeps it that way

A portal has six environment variables. That is the complete list, not an excerpt, and the shortness is the point: a portal that needs a seventh is usually a portal about to do something the api should be doing for it.

VARIABLE	CLASS	VALUE AND NOTES
VITE_API_BASE	public	https://api.atavishaala.in/api/v1 — the namespace is appended in code.
VITE_NAMESPACE	public	admin partner fleet. Determines the client namespace and cookie name.
VITE_SENTRY_DSN	public	Per-portal Sentry project.
VITE_POSTHOG_KEY	public	Omitted on admin: no autocapture on a PII-adjacent surface.
VITE_CLOUDINARY_CLOUD	public	For rendering signed URLs only. No key, no secret.
VITE_BUILD_SHA	public	Injected by CI. Shown in the footer for support triage.

The hard rule underneath the table: every value prefixed VITE_ is inlined into the JavaScript bundle at build time by Vite and is therefore published to anyone who opens the network tab. This is not a risk to be managed, it is a property of the build. There is no such thing as a secret VITE_ variable, and a value that is secret today does not become safe by being named carefully. Invariant I-13 states it as a rule the team is held to; the two checks below are what actually hold the line.

CHECK	WHERE	BEHAVIOUR
Name grep	Pre-commit hook and CI gate, three portal repos	A ripgrep pass for VITE_*(SECRET KEY TOKEN PASSWORD) across .env files, the workflow file, the Pages project variable dump committed as env.example, and source. A hit fails the commit locally and fails the build in CI. VITE_POSTHOG_KEY and VITE_SENTRY_DSN are the two allow-listed names, listed explicitly rather than pattern-matched, so a new KEY cannot slide in behind them.
Bundle grep	CI gate, three portal repos, post-build	The built assets are grepped for the literal prefixes of the secret formats actually in use — the Razorpay key id prefix, the Cloudinary API secret shape, and the PEM header. This catches the case the name grep cannot: a secret pasted into source as a bare string with no VITE_ name at all.
Secret scan	CI gate, all five repos	gitleaks over full history, as in section 1.7. This is the backstop, not the first line: by the time gitleaks finds a key it is already in every clone and must be rotated, not deleted.

Two of the six rows encode decisions rather than plumbing. VITE_POSTHOG_KEY is deliberately omitted on the admin portal: autocapture on a surface that displays masked PII and staff actions would send field values and page structure to a third party, and no amount of configuration makes that a good default. VITE_CLOUDINARY_CLOUD is the cloud name only — signing happens in the api and the portal receives a signed URL, so the portal never holds a Cloudinary key. VITE_BUILD_SHA is the diagnosis lever named in section 1.2.1: it is rendered in the footer of all three portals so that a support conversation can establish which build a user is on before anyone guesses at a kit version.

1.9 RELEASE, VERSIONING AND ROLLBACK

Semver for two packages, a manual bump, and one asymmetric rollback

Both published packages follow semver, but they mean slightly different things by it because one is generated and one is hand-written.

PACKAGE	MAJOR MEANS	MINOR MEANS	PATCH MEANS
@atavishaala/contract	A field was removed or narrowed, or an endpoint path or method changed. Requires a coordinated PR in every portal that touches the affected route.	A field, endpoint, enum member or error code was added. Existing calls still typecheck.	A description or example changed. No shape change at all.
@atavishaala/kit	A prop was removed or renamed, a component was deleted, or a default behaviour changed. Needs both FE reviews and a coordinated portal PR.	A component or an optional prop was added. Additive only, by the convention in section 1.6.	A bug fix with no API surface change.

There is no changesets tooling and no release-please bot. Both were considered and both lost to the same argument: they solve coordination across many packages and many contributors, and this is two packages and four developers. What they would add is a second thing to debug when a publish fails at 19:00 on a Wednesday. The manual protocol is two steps and one gate: bump the version field in package.json, add a dated CHANGELOG.md entry naming the task ID, and CI fails the publish workflow if the top CHANGELOG heading does not match the package version. That gate is cheap and it is the only reason the changelog stays honest.

Publishing goes to GitHub Packages under the atavishaala scope, private, on a tag push from main. Consumers authenticate with a repository-scoped token in Actions and a personal token in the developer .npmrc. Section 1.11 names the expiry of that token as a single point of failure, because it is one.

1.9.1 Rollback is not symmetric, and that shapes the migration policy

PORTAL ROLLBACK: INSTANT AND SAFE

Cloudflare Pages retains previous deployments and can promote any one of them to production from the dashboard in about ten seconds, with no rebuild and therefore no dependency on Actions minutes or on the package registry being reachable. A portal rollback restores an older bundle talking to the current api. Because the api only ever adds to its contract, the older bundle still typechecked against a subset of what the api now accepts, so it works. This is the primary rollback lever for anything user-visible and it needs no engineer with a terminal.

API ROLLBACK: NOT AVAILABLE IN PRACTICE

Cloud Run can route traffic back to a previous revision just as quickly, but the database has already moved. Migrations are forward-only — there are no down migrations in the repo, by policy — so revision N-1 may be running against a schema it has never seen. If M6 added a NOT NULL column with a default and revision N-1 does not write it, inserts fail. The honest position is that an api rollback is a schema decision, not a deploy button, and it must be reasoned about per migration.

That asymmetry is the whole reason migrations are additive-only. If an api rollback were safe, the team could afford destructive migrations and fix mistakes by reverting. It is not, so the policy compensates: add columns nullable or with a default, never drop or rename in the same release as the code that stops using them, and retire a column in a later week once no revision in the rollback window references it. AA-03 sets out the migration rules and the expand-contract sequence in full; this section only explains why they exist.

1.10 REPOSITORY BOOTSTRAP CHECKLIST

Day one, from template to a green no-op PR, in seven steps

This is the sequence FE-1 follows in task P1-FE1-01 to stand up a portal repository, and the sequence anybody follows if a fourth portal is ever needed. It takes about forty minutes per repository, most of it waiting. Do it in this order — the branch protection rule cannot be configured until at least one workflow run exists for the check names to be selectable.

#	STEP	WHAT YOU DO AND WHY	BY HAND
1	Create from template	Generate from atavishaala-portal-template: the Vite plus React plus TypeScript skeleton, the Tailwind preset importing from the kit, the eight custom ESLint rules already wired as errors, the workflow file, and an env.example listing all six VITE_ names with placeholder values. Rename the package, set VITE_NAMESPACE, delete the sample route.	Template
2	Create the Cloudflare Pages project	One project per repository, connected to the GitHub repo, build command npm run build, output dir dist. Add the custom subdomain from the table in section 1.4 and let Cloudflare issue the certificate. Set production branch to main so that every other branch produces a preview rather than a production deploy.	Manual
3	Add the four secrets	In the repository Actions secrets: NPM_READ_TOKEN for the atavishaala scope on GitHub Packages, CLOUDFLARE_API_TOKEN and CLOUDFLARE_ACCOUNT_ID for the deploy step, and SENTRY_AUTH_TOKEN for source-map upload. None of these is a VITE_ value and none of them reaches the bundle. Nothing else belongs here.	Manual
4	Add the branch protection rule	Protect main: require a pull request, require the ten blocking checks from section 1.7 by name, require branches to be up to date, and disallow force pushes. This is the step that turns the gates from advisory into real, and it is per-repository because the free tier has no organisation ruleset to inherit.	Manual
5	Add CODEOWNERS	Two lines: the whole repo to the pair of front-end developers, and any path under src/api or the workflow file additionally to the back-end reviewer, so a change to how a portal talks to the api cannot be self-merged by one person.	Template
6	Run npm run verify locally	The composite script: typecheck, lint, unit tests, build, bundle-size check. Getting this green on the untouched template before writing any feature code means the first red CI run is about your code and not about the scaffold.	Template
7	Open a no-op pull request	Change one line in the README on a chore/ branch and open a PR. Confirm every required check appears and passes, confirm the Pages preview URL is posted and loads, then confirm the merge button is genuinely blocked with a failing check by temporarily breaking a type. Merge, and confirm production deploys.	Manual

Four of the seven steps are marked by hand, and that is not an oversight to be automated later. The free tier has no organisation-level rulesets, no reusable secret store above the repository, and no Terraform state anybody is paying to host, so Pages project creation, secret entry and branch protection are dashboard work every time. Writing an infrastructure-as-code layer to automate three repositories that are created once would cost more than the four repetitions. The mitigation is that step 7 is not optional: the no-op PR is the only thing that proves the manual steps were actually done, which is why it includes deliberately breaking a type to see the merge button go red. AA-11 sequences all of this as week-1 tasks P1-FE1-01 and P1-FE2-01, with the working agreement and the gate log opened in P1-ALL-01.

THE API REPO BOOTSTRAP IS NOT IN THIS LIST

atavishaala-api already exists and is already deployed to Cloud Run; nothing here creates it. What week 1 adds to it is the backoffice module, the token-audience split and the idempotency middleware. AA-02 covers its internal layout and AA-03 its migration set.

1.11 WHAT COULD STILL GO WRONG HERE

Six failure modes in this layer, how you will notice, and what you do

These are failures of the repository and pipeline layer specifically. Application-level risks live in the risk register the programme charter carries; what follows is the set that a topology decision creates and that no amount of application care will prevent.

#	FAILURE MODE	DETECTION SIGNAL	RESPONSE
1	A portal ships with a stale @atavishaala/contract pin and 400s on every write. Reads keep working because the response shape is a superset, so the portal looks healthy until somebody tries to save something.	A cluster of 400 VALIDATION_FAILED responses on one namespace only, with the same missing field name in every log line, starting within minutes of a deploy. The api log carries the client namespace, so the blast radius is obvious from one query.	Roll the Pages deployment back to the previous production build (about ten seconds, section 1.9.1), then bump the contract pin and re-release properly. Do not hotfix the api to accept the old shape — that turns one skew into a permanent compatibility branch.
2	The GitHub Packages read token expires or is revoked. All three portal builds fail at the install step at the same moment, as does any developer running a clean npm ci. This is a single point of failure and is named as one: there is no second registry and no vendored copy of the kit.	Three simultaneous red builds across three repositories with an identical 401 Unauthorized from npm.pkg.github.com in the install log. If all three go red together, suspect the token before suspecting the code.	Mint a new token, update the secret in all three repositories and every local .npmrc. Mitigation in advance: record the token expiry date in the Monday sync notes and rotate it in week 5 rather than discovering it in week 9. Production is unaffected while this is broken — already deployed bundles keep serving — so this blocks shipping, not serving.
3	Cloudflare Pages monthly builds are exhausted. Every branch push produces a preview build, so four repositories in a busy week is roughly 120 builds and the 500 cap is a real ceiling, not a theoretical one.	Pages reports the build as failed with a quota message rather than a compile error, and the preview URL never appears on the PR. The AA-09 ledger alerts at 350 builds, which is the signal you should be reacting to instead.	Turn off preview deployments for branches whose name starts with chore/, and review production-only. Rolling back a production deploy still works while the cap is hit, because promoting a retained deployment is not a build. The paid gap is the Pages Pro tier at 5 000 builds for about 1 700 rupees a month; the workaround is fewer pushes per PR, which is free and slightly worse.
4	Two developers edit the kit in the same week and race the version bump. Both branches bump 1.4.0 to 1.5.0; the second merge publishes over an already-consumed version or the publish fails outright on a duplicate tag.	A publish workflow failing with a version-already-exists error, or worse, a portal whose lockfile integrity hash no longer matches the registry copy of a version it has already installed. The second is the dangerous one because it can present as an unexplained build failure in a repository nobody touched.	The bump is not in the feature PR. A kit feature PR changes source and CHANGELOG.md only; a separate one-line release PR sets the version and pushes the tag, and it is FE-1 who opens it. If a version was already published and consumed, never republish it — burn the number and publish the next patch.
5	A new repository is created without the branch protection rule, so its ten blocking gates run and are ignored. Everything looks configured because the checks appear on the PR.	A merge commit on main whose PR has a red check. This is invisible unless somebody looks, which is why step 7 of section 1.10 requires deliberately breaking a type to watch the merge button go red.	Add the rule, then audit main for merges past a failing check. Longer term the only honest mitigation on this tier is that repository creation is a two-person step: FE-1 creates, FE-2 opens the no-op PR and confirms the block.
6	A secret reaches a shipped bundle despite the checks — pasted as a bare literal with no VITE_ name, into a file the bundle grep pattern does not cover.	Nothing automatic. This one is found by the secret scan on a later commit, by the provider, or by somebody reading the deployed JavaScript. Assume detection is late.	Rotate the credential first, then remove it, in that order. Removing the commit does not help — the bundle is on a CDN and in browser caches. The gates in section 1.8 reduce the probability; they do not make this impossible, and the response plan exists because it is not impossible.

Note what these six have in common. Five of them block shipping and none of them takes production down, which is the intended shape of a topology risk: the cost of the five-repository split is paid in delivery friction, not in availability. The one exception is failure mode 1, where a stale pin does reach users, and it is also the one with the fastest and most reliable recovery. That asymmetry was deliberate when the split was chosen in AA-00 section 0.3.1, and it is the reason the version-skew tax of section 1.2.1 was accepted rather than argued away.

WHAT THIS DOCUMENT DOES NOT COVER

The internal structure of atavishaala-api and its module boundaries are in AA-02. The migration set and the expand-contract rules that make a portal rollback safe are in AA-03. The CORS list, the cookie posture and the PII disclosure contract are in AA-08. Free-tier limits, alert thresholds and the priced paid gaps are in AA-09. The week-by-week task sequence that turns this topology into working repositories is in AA-11.